

PROGETTO TRENTO EASY START

Titolo del documento:

Trento Easy Start - Documento di Sviluppo (Deliverable 3)

Document Info

Doc. Name	D3-TrentoEasyStart_Sviluppo
Doc. Number	D3 V1.0
Description	Documento di sviluppo del progetto Trento Easy Start

INDICE

- 1. [Scopo del documento](#)
- 2. [User Stories](#)
- 3. [User Flow](#)
- 4. [Web APIs](#)
- 5. [Implementation](#)
 - [Repository Organization](#)
 - [Branching Strategy e Organizzazione del Lavoro](#)
 - [Dependencies](#)
 - [Database](#)
 - [Testing](#)
- 6. [FrontEnd](#)
- 7. [Deployment](#)

Scopo del documento

Il presente documento riporta tutte le informazioni necessarie per lo sviluppo del sistema "Trento Easy Start". In particolare, presenta le user stories, gli user flow, la documentazione delle Web APIs, l'organizzazione del repository, le strategie di branching, le dipendenze, il modello dati, le pratiche di testing, lo sviluppo del front-end e gli aspetti di deployment. Questo documento serve come guida dettagliata per il team di sviluppo, garantendo che tutte le funzionalità necessarie siano implementate in modo efficace e coerente.

2. User Stories

Nel presente capitolo vengono riportate le user stories individuate per il progetto "Trento Easy Start", partendo dai requisiti definiti nella fase di analisi e progettazione.

User Story 1: Registrazione e Autenticazione Utente

Come nuovo residente di Trento, **voglio** registrarmi e autenticarmi sulla piattaforma, in modo da poter accedere ai servizi offerti.

Criteri di accettazione:

- Un utente può registrarsi fornendo nome, email e password.
- Un utente può autenticarsi utilizzando le proprie credenziali.
- Il sistema invia un messaggio su telegram di conferma alla registrazione e all'accesso.
- Il sistema gestisce i messaggi di errore per credenziali errate o email già registrate.

Tasks – User Story 1:

1. Implementare il form di registrazione con campi nome, email e password.
2. Integrare l'autenticazione tramite nome utente e password
3. Configurare l'invio del messaggio di conferma tramite un bot telegram.
4. Gestire i messaggi di errore per email già registrate e credenziali errate.
5. Testare il processo di registrazione e autenticazione.

User Story 2: Ricerca e Prenotazione Alloggi

Come residente di Trento, **voglio** cercare e prenotare alloggi disponibili utilizzando vari criteri di ricerca, in modo da trovare facilmente l'alloggio che soddisfa le mie esigenze.

Criteri di accettazione:

- L'utente può inserire uno o più criteri di ricerca (località, prezzo).
- Il sistema restituisce una lista di alloggi che corrispondono ai criteri inseriti.
- Ogni alloggio mostra dettagli come titolo, tipo, descrizione, località, prezzo e immagini.
- L'utente può prenotare un alloggio dalla lista dei risultati.
- Se non vengono trovati alloggi, viene mostrato un messaggio informativo.

Tasks – User Story 2:

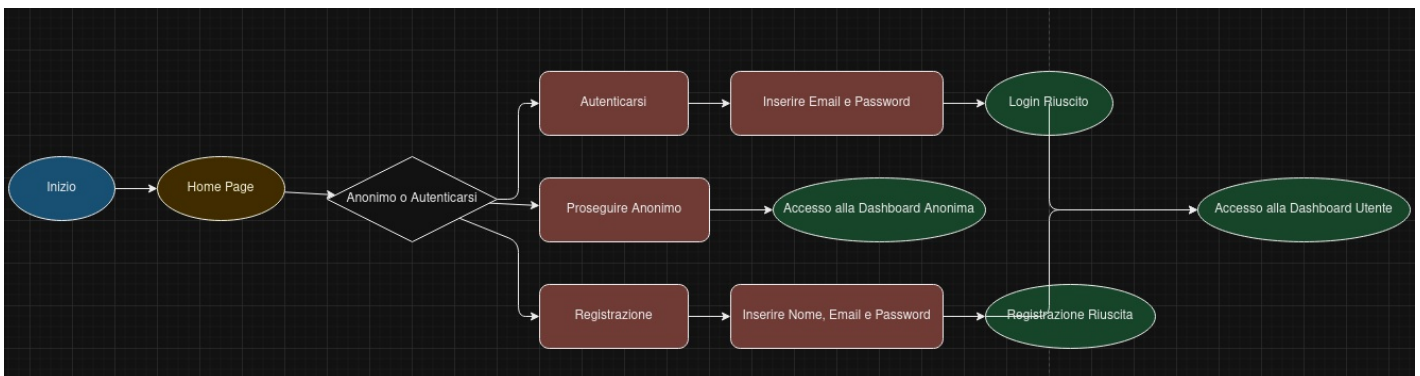
1. Creare il campo di ricerca nella UI con opzioni per località, tipo e prezzo.
2. Implementare la logica di ricerca nel backend per gestire i vari criteri.
3. Recuperare e visualizzare i risultati della ricerca nel frontend.
4. Implementare la funzionalità di prenotazione alloggio aprendo il relativo sito internet.
5. Gestire i casi in cui non vengono trovati alloggi corrispondenti ai criteri.
6. Testare la funzionalità di ricerca e prenotazione con diversi criteri e combinazioni.

3. User Flow

In questa sezione vengono descritti gli "user flows" per i diversi ruoli all'interno della piattaforma "Trento Easy Start".

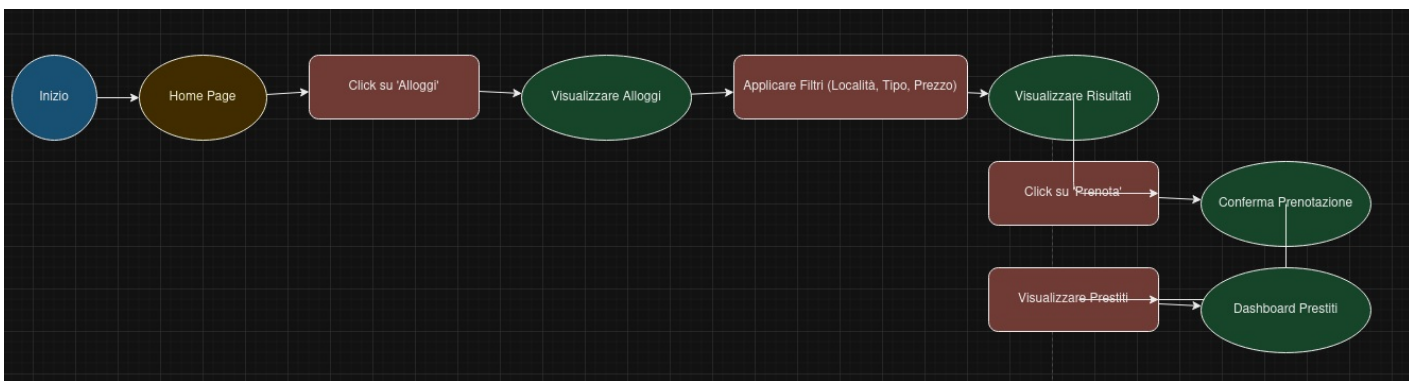
User Flow: Registrazione e Autenticazione Utente

Figura 1 descrive lo user flow relativo alla registrazione e all'autenticazione degli utenti nella piattaforma "Trento Easy Start". L'utente può scegliere tra la registrazione tramite credenziali locali o tramite autenticazione Google. Dopo la registrazione, l'utente riceve un'email di conferma e può accedere alla propria dashboard.



User Flow: Ricerca e Prenotazione Alloggi

Figura 2 descrive lo user flow relativo alla ricerca e prenotazione degli alloggi nella piattaforma "Trento Easy Start". L'utente seleziona la sezione "Alloggi", applica i filtri desiderati, visualizza i risultati e procede alla prenotazione dell'alloggio scelto.



4. Web APIs

Le API del sistema "Trento Easy Start" sono state documentate secondo le specifiche OpenAPI 3.0. Di seguito viene riportato il file YAML che descrive le endpoint disponibili, i modelli di dati e le risposte del sistema. La pagina con swagger si può consultare a <https://trento-easy-start-1.onrender.com/api-docs/>

Specifica OpenAPI 3.0

C

```
openapi: 3.0.0
info:
  title: Trento Easy Start API
  description: API per gestire le funzionalità di Trento Easy Start, inclusi autenticazione, alloggi, contatti, note
  version: 1.0.0
servers:
  - url: http://localhost:5000/api
    description: Server di sviluppo locale

components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

  schemas:
    User:
      type: object
      required:
        - name
        - email
        - password
      properties:
        id:
          type: string
          description: ID univoco dell'utente
          example: "60d0fe4f5311236168a109ca"
        name:
          type: string
          description: Nome dell'utente
          example: "Mario Rossi"
        email:
          type: string
          format: email
          description: Email dell'utente
          example: "mario.rossi@example.com"
        password:
          type: string
          format: password
          description: Password dell'utente
          example: "password123"
        date:
```

type: string
format: date-time
description: Data di registrazione
example: "2023-04-05T14:30:00Z"

Accommodation:

type: object
required:
- title
- type
- description
- location
- price
properties:
id:
type: string
description: ID univoco dell'alloggio
example: "60d0fe4f5311236168a109cb"
title:
type: string
description: Titolo dell'alloggio
example: "Appartamento Centrale"
type:
type: string
enum: [Affitti Brevi, Affitti Lunghi, Case Vacanza]
description: Tipo di alloggio
example: "Affitti Brevi"
description:
type: string
description: Descrizione dell'alloggio
example: "Appartamento moderno nel centro di Trento."
location:
type: string
description: Località dell'alloggio
example: "Trento, Italia"
price:
type: number
format: float
description: Prezzo dell'alloggio
example: 750.00
images:
type: array
items:
type: string
format: uri
description: Elenco di URL delle immagini dell'alloggio
example: ["http://example.com/image1.jpg", "http://example.com/image2.jpg"]
datePosted:
type: string
format: date-time
description: Data di pubblicazione
example: "2024-01-15T10:00:00Z"
postedBy:
type: string
description: ID dell'utente che ha pubblicato l'alloggio
example: "60d0fe4f5311236168a109ca"

Contact:

type: object
required:
- name
- email
- subject
- message
properties:
id:
type: string
description: ID univoco del contatto
example: "60d0fe4f5311236168a109cc"
name:
type: string

description: Nome del contatto
example: "Luigi Bianchi"
email:
type: string
format: email
description: Email del contatto
example: "luigi.bianchi@example.com"
subject:
type: string
description: Oggetto del messaggio
example: "Richiesta Informazioni"
message:
type: string
description: Messaggio del contatto
example: "Vorrei avere maggiori informazioni sugli alloggi disponibili."
dateSent:
type: string
format: date-time
description: Data di invio del messaggio
example: "2024-01-16T09:15:00Z"

AuthResponse:
type: object
properties:
token:
type: string
description: JWT per l'autenticazione
example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

ErrorResponse:
type: object
properties:
msg:
type: string
description: Messaggio di errore
example: "Server Error"

Event:
type: object
properties:
id:
type: string
description: ID univoco dell'evento
example: "event123"
title:
type: string
description: Titolo dell'evento
example: "Concerto Jazz"
description:
type: string
description: Descrizione dell'evento
example: "Un concerto di jazz nel centro storico di Trento."
date:
type: string
format: date-time
description: Data dell'evento
example: "2024-05-20T20:00:00Z"
link:
type: string
format: uri
description: Link per maggiori informazioni
example: "http://example.com/evento123"

News:
type: object
properties:
id:
type: string
description: ID univoco della notizia
example: "news123"
title:

```
    type: string
    description: Titolo della notizia
    example: "Apertura Nuovo Parco"
  description:
    type: string
    description: Descrizione della notizia
    example: "Il Comune di Trento ha inaugurato un nuovo parco nel centro città."
  date:
    type: string
    format: date-time
    description: Data della notizia
    example: "2024-03-10T12:00:00Z"
  link:
    type: string
    format: uri
    description: Link per maggiori informazioni
    example: "http://example.com/notizia123"
  type:
    type: string
    enum: [avviso]
    description: Tipo di notizia
    example: "avviso"
```

Service:

```
  type: object
  properties:
    id:
      type: string
      description: ID univoco del servizio
      example: "service123"
    title:
      type: string
      description: Titolo del servizio
      example: "Trasporti"
    description:
      type: string
      description: Descrizione del servizio
      example: "Informazioni sui trasporti pubblici a Trento."
    category:
      type: string
      description: Categoria del servizio
      example: "Trasporti"
    link:
      type: string
      format: uri
      description: Link per maggiori informazioni
      example: "http://example.com/trasporti"
```

responses:

```
  UnauthorizedError:
    description: Accesso negato. Token mancante o non valido.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
```

```
  NotFoundError:
    description: Risorsa non trovata.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
```

```
  ServerError:
    description: Errore del server.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
```

paths:

```
/auth/register:
  post:
    summary: Registrazione di un nuovo utente
    tags:
      - Auth
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - name
              - email
              - password
            properties:
              name:
                type: string
                example: "Mario Rossi"
              email:
                type: string
                format: email
                example: "mario.rossi@example.com"
              password:
                type: string
                format: password
                example: "password123"
    responses:
      '200':
        description: Utente registrato con successo
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/AuthResponse'
      '400':
        description: Richiesta non valida o utente già registrato
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'
      '500':
        $ref: '#/components/responses/ServerError'

/auth/login:
  post:
    summary: Login di un utente
    tags:
      - Auth
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required:
              - email
              - password
            properties:
              email:
                type: string
                format: email
                example: "mario.rossi@example.com"
              password:
                type: string
                format: password
                example: "password123"
    responses:
      '200':
        description: Login effettuato con successo
        content:
          application/json:
```

```
    schema:
      $ref: '#/components/schemas/AuthResponse'
  '400':
    description: Credenziali non valide
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  '500':
    $ref: '#/components/responses/ServerError'
```

/auth:

```
get:
  summary: Ottieni i dati dell'utente autenticato
  tags:
    - Auth
  security:
    - BearerAuth: []
  responses:
    '200':
      description: Dati utente recuperati con successo
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/User'
    '401':
      $ref: '#/components/responses/UnauthorizedError'
    '500':
      $ref: '#/components/responses/ServerError'
```

/accommodations:

```
post:
  summary: Crea un nuovo alloggio
  tags:
    - Accommodations
  security:
    - BearerAuth: []
  requestBody:
    required: true
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Accommodation'
  responses:
    '200':
      description: Alloggio creato con successo
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Accommodation'
    '400':
      description: Richiesta non valida
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
    '401':
      $ref: '#/components/responses/UnauthorizedError'
    '500':
      $ref: '#/components/responses/ServerError'
```

get:

```
  summary: Ottieni tutti gli alloggi con filtri opzionali
  tags:
    - Accommodations
  parameters:
    - in: query
      name: location
      schema:
        type: string
      description: Filtra per località
```



```

    example: "Trento"
  - in: query
    name: type
    schema:
      type: string
      enum: [Affitti Brevi, Affitti Lunghi, Case Vacanza]
    description: Filtra per tipo di alloggio
    example: "Affitti Brevi"
  - in: query
    name: minPrice
    schema:
      type: number
      format: float
    description: Prezzo minimo
    example: 500
  - in: query
    name: maxPrice
    schema:
      type: number
      format: float
    description: Prezzo massimo
    example: 1500

```

responses:

```

'200':
  description: Elenco di alloggi
  content:
    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/Accommodation'
'500':
  $ref: '#/components/responses/ServerError'

```

/accommodations/{id}:

get:

summary: Ottieni un alloggio per ID

tags:

- Accommodations

parameters:

```

- in: path
  name: id
  required: true
  schema:
    type: string
  description: ID dell'alloggio

```

responses:

```

'200':
  description: Dettagli dell'alloggio
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/Accommodation'
'404':
  $ref: '#/components/responses/NotFoundError'
'500':
  $ref: '#/components/responses/ServerError'

```

put:

summary: Aggiorna un alloggio per ID

tags:

- Accommodations

security:

- BearerAuth: []

parameters:

```

- in: path
  name: id
  required: true
  schema:
    type: string
  description: ID dell'alloggio

```

```
requestBody:
  required: true
  content:
    application/json:
      schema:
        $ref: '#/components/schemas/Accommodation'
responses:
  '200':
    description: Alloggio aggiornato con successo
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Accommodation'
  '400':
    description: Richiesta non valida
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  '401':
    $ref: '#/components/responses/UnauthorizedError'
  '404':
    $ref: '#/components/responses/NotFoundError'
  '500':
    $ref: '#/components/responses/ServerError'
```

```
delete:
  summary: Elimina un alloggio per ID
  tags:
    - Accommodations
  security:
    - BearerAuth: []
  parameters:
    - in: path
      name: id
      required: true
      schema:
        type: string
      description: ID dell'alloggio
  responses:
    '200':
      description: Alloggio eliminato con successo
      content:
        application/json:
          schema:
            type: object
            properties:
              msg:
                type: string
                example: "Alloggio eliminato"
    '401':
      $ref: '#/components/responses/UnauthorizedError'
    '404':
      $ref: '#/components/responses/NotFoundError'
    '500':
      $ref: '#/components/responses/ServerError'
```

```
/contact:
  post:
    summary: Invia un messaggio di contatto
    tags:
      - Contact
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/Contact'
    responses:
      '200':
        description: Messaggio inviato con successo
```

```
    content:
      application/json:
        schema:
          type: object
          properties:
            msg:
              type: string
              example: "Messaggio inviato con successo"
  '400':
    description: Richiesta non valida
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/ErrorResponse'
  '500':
    $ref: '#/components/responses/ServerError'
```

get:

```
summary: Ottieni tutti i messaggi di contatto
tags:
  - Contact
security:
  - BearerAuth: []
responses:
  '200':
    description: Elenco dei messaggi di contatto
    content:
      application/json:
        schema:
          type: array
          items:
            $ref: '#/components/schemas/Contact'
  '401':
    $ref: '#/components/responses/UnauthorizedError'
  '500':
    $ref: '#/components/responses/ServerError'
```

/external-accommodations/fetch:

get:

```
summary: Recupera annunci di alloggio esterni con filtri
tags:
  - External Accommodations
parameters:
  - in: query
    name: location
    schema:
      type: string
    description: Località per la ricerca
    example: "trento"
  - in: query
    name: type
    schema:
      type: string
      enum: [Affitti Brevi, Affitti Lunghi, Case Vacanza]
    description: Tipo di alloggio
    example: "Affitti Brevi"
  - in: query
    name: minPrice
    schema:
      type: number
      format: float
    description: Prezzo minimo
    example: 500
  - in: query
    name: maxPrice
    schema:
      type: number
      format: float
    description: Prezzo massimo
    example: 1500
responses:
```

```
'200':
  description: Elenco degli annunci di alloggio esterni
  content:
    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/Accommodation'
'500':
  $ref: '#/components/responses/ServerError'
```

/news:

```
get:
  summary: Ottieni tutte le notizie
  tags:
    - News
  responses:
    '200':
      description: Elenco delle notizie
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/News'
    '404':
      $ref: '#/components/responses/NotFoundError'
    '500':
      $ref: '#/components/responses/ServerError'
```

/events:

```
get:
  summary: Ottieni tutti gli eventi
  tags:
    - Events
  responses:
    '200':
      description: Elenco degli eventi
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/Event'
    '404':
      $ref: '#/components/responses/NotFoundError'
    '500':
      $ref: '#/components/responses/ServerError'
```

/services:

```
get:
  summary: Ottieni tutti i servizi
  tags:
    - Services
  responses:
    '200':
      description: Elenco dei servizi
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/Service'
    '404':
      $ref: '#/components/responses/NotFoundError'
    '500':
      $ref: '#/components/responses/ServerError'
```

security:

```
- BearerAuth: []
```

5. Implementation

L'applicazione è stata sviluppata utilizzando le seguenti tecnologie: NodeJS per il backend, VueJS per il frontend, e MongoDB per la gestione dei dati. Lo stack tecnologico è stato scelto per la sua flessibilità, scalabilità e facilità di integrazione con le tecnologie moderne.

5.1 Repository Organization

Il codice del progetto (<https://github.com/ismacarbo/TrentoEasyStart>) è organizzato secondo la seguente struttura:

```
TrentoEasyStart
├── /swagger_code
│   └── openapi3.yaml
├── /trentoeasystart-backend
│   ├── /api
│   ├── /middleware
│   ├── /models
│   ├── app.js
│   ├── package.json
│   ├── .env.example
│   └── README.md
├── /trentoeasystart-frontend
│   ├── /src
│   ├── /public
│   ├── package.json
│   └── README.md
├── /images
│   ├── UserFlow_Registrazione_Auth.png
│   ├── UserFlow_Ricerca_Prenotazione.png
│   ├── UserFlow_Gestione_Prestiti.png
│   ├── Home_Page.png
│   ├── Gestione_Alloggi.png
│   ├── Ricerca_Alloggi.png
│   └── Invio_Contatto.png
├── .gitignore
└── README.md
```

5.2 Branching Strategy e Organizzazione del Lavoro

Lo sviluppo ha visto una collaborazione attiva tra i membri del team.

- **Master:** Contiene il codice sempre stabile e pronto per la produzione.
- **Develop:** Branch principale per lo sviluppo delle nuove funzionalità.
- **Deliverable:** Branch che contiene i deliverable.

5.3 Dependencies

Il progetto si basa sui seguenti moduli esterni:

Backend Dependencies:

- **Cors:** Supporto per le richieste CORS.
- **Express:** Framework web per il backend.
- **Jsonwebtoken:** Gestione dell'autenticazione JWT.
- **Mongoose:** Libreria per interfacciarsi con MongoDB.

Development Dependencies:

- **Dotenv:** Gestione delle variabili d'ambiente.

5.4 Database

Per la gestione dei dati abbiamo utilizzato MongoDB. Le principali collezioni definite sono:

- **Users:** Memorizza i dati degli utenti registrati.
- **Accommodations:** Contiene gli alloggi disponibili.
- **Contacts:** Memorizza i messaggi di contatto inviati dagli utenti.
- **News:** Contiene le notizie recuperate dalle API esterne.
- **Events:** Memorizza gli eventi recuperati dalle API esterne.
- **Services:** Contiene i servizi offerti dal Comune di Trento.
- **Loans:** Memorizza i prestiti degli utenti.

5.5 Testing

Le API e il relativo codice presentano una suite di test che consente di verificarne il corretto funzionamento. I test sono eseguiti utilizzando Jest e Supertest, e integrati nel processo di CI/CD tramite GitHub Actions.

I test sono organizzati in file `test.js` affiancati ai rispettivi file di implementazione nel backend. Attualmente, sono stati implementati 50 casi di test, coprendo le principali funzionalità del sistema.

Casi di Test

N.	Test Case	Descrizione Test Case	Test Data	Precondizioni	Dipendenze	Risultato Atteso	Risultato Riscontrato	Note
1	Creazione di un'account in modo corretto	L'utente fornisce uno username non vuoto e una password che rispetta le politiche di sicurezza.	• Username: mario.rossi • Password: Password@123	• Lo username non è mai stato inserito prima nel sistema.	---	L'account viene creato con successo. Il sistema risponde con un messaggio di conferma.	---	---
2	Creazione di un'account specificando uno username già esistente	L'utente tenta di registrarsi utilizzando uno username già presente nel sistema.	• Username: mario.rossi • Password: Password@123	• Lo username è già stato registrato.	Test Case 1	Viene mostrato un messaggio di errore " User exists ", l'account non viene creato e vengono suggeriti alcuni	---	Questo caso di test deve essere eseguito dopo il Test Case 1.

			3			alcuni username disponibili .		
3	Creazione di un'account non specificando lo username	L'utente tenta di registrarsi senza fornire uno username.	- User name: (vuoto) - Password: Password @123	---	---	Viene mostrato un messaggio di errore "Empty user" e l'account non viene creato.	---	---
4	Creazione di account violando le politiche di sicurezza (password banale)	L'utente fornisce una password che non rispetta le politiche di sicurezza.	- User name: luca.bianchi - Password: password	---	---	Viene mostrato un messaggio di errore "Password does not meet security policies" e l'account non viene creato.	---	---
5	Creazione di account con conferma password errata	L'utente fornisce una conferma password che non corrisponde alla password iniziale.	- User name: anna.verdi - Password: Password @123 - Conferma Password: Password @124	---	---	Viene mostrato un messaggio di errore "Passwords do not match" e l'account non viene creato.	---	---

6	Login con credenziali corrette	L'utente fornisce email e password validi per l'autenticazione.	- Email: mario.rossi@example.com - Password: Password@123	Il cont o utent e è già stato creat o (Test Case 1).	Test Case 1	Viene restituito un token JWT valido e l'utente accede alla dashboard.	---	---
7	Login con email inesistente	L'utente tenta di autenticarsi con un'email non registrata.	- Email: non.esistente@example.com - Password: Password@123	---	---	Viene mostrato un messaggio di errore "Invalid credentials" e l'utente non accede.	---	---
8	Login con password errata	L'utente tenta di autenticarsi con una password sbagliata.	- Email: mario.rossi@example.com - Password: WrongPassword@123	Il cont o utent e è già stato creat o (Test Case 1).	Test Case 1	Viene mostrato un messaggio di errore "Invalid credentials" e l'utente non accede.	---	---
9	Accesso ai dati	L'utente tenta di	---	---	---	Viene mostrato	---	---

	utente senza token	accedere ai propri dati senza fornire un token JWT.				un messaggio di errore "Unauthorized" .		
10	Accesso ai dati utente con token non valido	L'utente tenta di accedere ai propri dati utilizzando un token JWT non valido o scaduto.	Token: invalid.token.string	---	---	Viene mostrato un messaggio di errore "Unauthorized" .	---	---
11	Creazione di un alloggio con dati validi	L'utente fornisce tutti i campi richiesti per creare un nuovo alloggio.	- Titolo: "Appartamento Centrale" - Tipo: "Affitti Brevi" - Descrizione: "Appartamento moderno nel centro di Trento." - Località: "Trento, Italia" - Prezzo: 750.00 - Immagini	Utente autenticato con privilegi di creazione alloggi.	---	L'alloggio viene creato con successo e viene restituito l'oggetto dell'alloggio creato.	---	---

			: ["http://example.com/image1.jpg", "http://example.com/image2.jpg"]					
12	Creazione di un alloggio mancante di campi obbligatori	L'utente tenta di creare un alloggio senza specificare tutti i campi obbligatori.	• Titolo: "Appartamento Nord" • Tipo: "Affitti Lunghi" • Descrizione: "Appartamento spazioso."	• Utente autenticato con privilegi di creazione alloggi.	---	Viene mostrato un messaggio di errore "Missing required fields" e l'alloggio non viene creato.	---	---
13	Recupero di tutti gli alloggi con filtri applicati	L'utente applica filtri di ricerca (località, tipo, prezzo) per recuperare gli alloggi.	• Località: "Trento" • Tipo: "Affitti Brevi" • Prezzo minimo: 500	• Esistono alloggi che corrispondono ai criteri di ricerca.	---	Viene restituita una lista di alloggi che corrispondono ai filtri applicati.	---	---

			• Prezzo massimo: 1500					
14	Recupero di tutti gli alloggi senza applicare filtri	L'utente recupera tutti gli alloggi disponibili senza applicare alcun filtro.	---	---	---	Viene restituita una lista completa di tutti gli alloggi disponibili .	---	---
15	Recupero di un alloggio per ID valido	L'utente richiede i dettagli di un alloggio specifico utilizzando un ID valido.	• ID: "60d0fe4f5311236168a109cb"	• L'alloggio con l'ID specificato esiste nel sistema.	---	Viene restituito l'oggetto dell'alloggio con tutti i dettagli.	---	---
16	Recupero di un alloggio per ID non valido	L'utente richiede i dettagli di un alloggio utilizzando un ID non valido.	• ID: "invalidID123"	---	---	Viene mostrato un messaggio di errore "Not Found" .	---	---
17	Aggiornamento di un alloggio con dati validi	L'utente aggiorna i dettagli di un alloggio esistente con dati corretti.	• ID: "60d0fe4f5311236168a109cb" • Titolo: "Appartamento Centrale Aggi	• Utente autenticato con privilegi di aggiornamento alloggi. • L'alloggio	---	L'alloggio viene aggiornato con successo e vengono restituiti i nuovi dettagli.	---	---

			orna to" • Prezzo: 800. 00	con l'ID speci ficat o esist e.				
18	Aggiorna mento di un alloggio con ID non esistente	L'utente tenta di aggiornar e un alloggio utilizzand o un ID che non esiste nel sistema.	• ID: "non exist entID123 " • Titolo: "App arta ment o Inesi stent e" • Prezzo: 900. 00	---	---	Viene mostrato un messaggi o di errore "Not Found" e l'alloggio non viene aggiornat o.	---	---
19	Eliminazio ne di un alloggio con ID valido	L'utente elimina un alloggio esistente utilizzand o un ID valido.	• ID: "60d 0fe4f 5311 2361 68a1 09cb "	• Uten te aute ntica to con privil egi di elimi nazio ne allog gi. • L'allo ggio con l'ID speci ficat o esist e.	---	L'alloggio viene eliminato con successo e viene restituito un messaggi o di conferma.	---	---
20	Eliminazio	L'utente		---	---	Viene	---	---

	ne di un alloggio con ID non valido	tenta di eliminare un alloggio utilizzando un ID non valido.	ID: "invalidDeleteID123"			mostrato un messaggio di errore "Not Found" e l'alloggio non viene eliminato.		
21	Invio di un messaggio di contatto con dati validi	L'utente invia un messaggio di contatto fornendo tutti i campi richiesti.	- Nome: "Luigi Bianchi" - Email: "luigi.bianchi@example.com" - Oggetto: "Richiesta Informazioni" - Messaggio: "Vorrei avere maggiori informazioni sugli alloggi disponibili."	---	---	Viene mostrato un messaggio di conferma "Messaggio inviato con successo" .	---	---
22	Invio di un messaggio di contatto mancante	L'utente tenta di inviare un messaggio di	Nome: "Giulia	---	---	Viene mostrato un messaggio di errore	---	---

	manca di campi obbligatori	o di contatto senza fornire uno o più campi obbligatori.	Verdi" Email: "giulia.verdi@example.com"			o di errore "Missing required fields" e il messaggio non viene inviato.		
23	Invio di un messaggio di contatto con email non valida	L'utente tenta di inviare un messaggio di contatto utilizzando un'email non valida.	- Nome: "Marco Rossi" - Email: "invalid-email-format" - Oggetto: "Problema Tecnico" - Messaggio: "Ho riscontrato un problema tecnico nell'applicazione."	---	---	Viene mostrato un messaggio di errore "Invalid email format" e il messaggio non viene inviato.	---	---
24	Recupero di tutti i messaggi di contatto come amministratore	L'amministratore richiede di recuperare tutti i messaggi di contatto.	---	Utente autenticato come amministratore	---	Viene restituita una lista completa di tutti i messaggi di contatto.	---	---

	amministratore	di contatto inviati dagli utenti.		e amministratore.		di contatto.		
25	Recupero di tutti i messaggi di contatto come utente non autorizzato	Un utente normale tenta di recuperare i messaggi di contatto.	---	• Utente autenticato senza privilegi di amministratore.	---	Viene mostrato un messaggio di errore "Unauthorized" .	---	---
26	Recupero di notizie tramite API	L'utente richiede di recuperare tutte le notizie disponibili tramite le API.	---	---	---	Viene restituita una lista completa di tutte le notizie.	---	---
27	Recupero di eventi tramite API	L'utente richiede di recuperare tutti gli eventi disponibili tramite le API.	---	---	---	Viene restituita una lista completa di tutti gli eventi.	---	---
28	Recupero di servizi tramite API	L'utente richiede di recuperare tutti i servizi disponibili tramite le API.	---	---	---	Viene restituita una lista completa di tutti i servizi.	---	---
29	Recupero di alloggi esterni con filtri applicati	L'utente applica filtri di ricerca per recuperare alloggi	• Località: "Trento" • Tipo: "Cas"	---	---	Viene restituita una lista di alloggi esterni che	---	---

		esterni.	• Prezzo minimo: 600 • Prezzo massimo: 1600			corrispondono ai filtri applicati.		
30	Recupero di alloggi esterni senza filtri	L'utente recupera tutti gli alloggi esterni disponibili senza applicare alcun filtro.	---	---	---	Viene restituita una lista completa di tutti gli alloggi esterni disponibili.	---	---
31	Prenotazione di un alloggio con dati validi	L'utente prenota un alloggio fornendo tutti i dati richiesti.	• ID Alloggio: "60d0fe4f5311236168a109cb" • Data inizio: "2024-06-01" • Data fine: "2024-06-10"	• Alloggio disponibile per le date selezionate. • Utenze autenticate.	---	La prenotazione viene creata con successo e viene restituito un messaggio di conferma.	---	---
32	Prenotazione di un alloggio con date non disponibili	L'utente tenta di prenotare un alloggio per date già	• ID Alloggio: "60d0fe4f5311	• Alloggio già prenotato per	Test Case 31	Viene mostrato un messaggio di errore " Dates not	---	---

		già occupate.	236168a109cb" - Data inizio: "2024-06-05" - Data fine: "2024-06-15"	le date selezionate. Uten te autenticato.		not available" e la prenotazione non viene creata.		
33	Prenotazione di un alloggio senza autenticazione	L'utente tenta di prenotare un alloggio senza essere autenticato.	- ID Alloggio: "60d0fe4f5311236168a109cb" - Data inizio: "2024-07-01" - Data fine: "2024-07-10"	---	---	Viene mostrato un messaggio di errore "Unauthorized" e la prenotazione non viene creata.	---	---
34	Rinnovo di un prestito con dati validi	L'utente rinnova un prestito esistente fornendo una nuova data di fine valida.	- ID Prestito: "loan123" - Nuova Data Fine: "2024-07-20"	- Prestito esistente e attivo. - Uten te autenticato.	---	Il prestito viene aggiornato con la nuova data di fine e viene restituito un messaggio di conferma.	---	---
35	Rinnovo di	L'utente	ID	Prest	---	Viene	---	---

	un prestito con nuova data di fine precedente e alla data corrente	tenta di rinnovare un prestito impostando una nuova data di fine già passata.	-- Prestito: "loan 123" • Nuova Data Fine: "2023-12-31"	-- prestito esistente e attivo. • Utente autenticato.		mostrato un messaggio di errore " Invalid end date " e il prestito non viene aggiornato.		
36	Terminazione anticipata di un prestito	L'utente termina un prestito prima della data di scadenza.	• ID Prestito: "loan 123"	• Prestito esistente e attivo. • Utente autenticato.	---	Il prestito viene terminato con successo e lo stato viene aggiornato a "Terminated". Viene mostrato un messaggio di conferma.	---	---
37	Terminazione di un prestito già terminato	L'utente tenta di terminare un prestito che è già stato terminato.	• ID Prestito: "loan 123"	• Prestito esistente e già terminato. • Utente autenticato.	---	Viene mostrato un messaggio di errore " Loan already terminated " e lo stato del prestito non viene modificato.	---	---
38	Visualizzazione dei prestiti attivi per un utente	L'utente visualizza la lista dei propri prestiti attivi.	---	• Utente autenticato con prestiti attivi.	---	Viene mostrata una lista di tutti i prestiti attivi dell'utente con i dettagli necessari.	---	---

39	Visualizzazione dei prestiti per un utente senza prestiti attivi	L'utente visualizza la lista dei propri prestiti quando non ne ha di attivi.	---	• Utente autenticato senza prestiti attivi.	---	Viene mostrato un messaggio informativo " No active loans found ".	---	---
40	Creazione di un alloggio con immagini non valide	L'utente tenta di creare un alloggio fornendo URL di immagini non valide.	• Titolo: "Villa Mare" • Tipo: "Case Vacanza" • Descrizione: "Villa con vista mare." • Località: "Riva del Garda, Italia" • Prezzo: 1200.00 • Immagini: ["invalid-url", "http://example.com/im	• Utente autenticato con privilegi di creazione alloggi.	---	Viene mostrato un messaggio di errore " Invalid image URLs " e l'alloggio non viene creato.	---	---

			age2 .jpg"]					
41	Recupero delle notizie con tipo diverso da "avviso"	L'utente richiede di recuperare notizie filtrando per tipo diverso da "avviso".	Tipo: "informazione"	---	---	Viene restituito una lista di notizie filtrate per il tipo specificato.	---	---
42	Recupero di un evento con ID valido	L'utente richiede i dettagli di un evento specifico utilizzando un ID valido.	ID Evento: "event123"	L'evento con l'ID specificato esiste.	---	Viene restituito l'oggetto dell'evento con tutti i dettagli.	---	---
43	Recupero di un evento con ID non valido	L'utente tenta di recuperare un evento utilizzando un ID non valido.	ID Evento: "invalidEventID123"	---	---	Viene mostrato un messaggio di errore "Not Found" .	---	---
44	Creazione di un prestito con dati validi	L'utente crea un nuovo prestito fornendo tutti i dati richiesti.	- Titolo: "Prestito Libro" - Data inizio: "2024-06-01" - Data fine: "2024-06-15"	Utente autenticato.	---	Il prestito viene creato con successo e viene restituito l'oggetto del prestito creato.	---	---
45	Creazione di un prestito	L'utente tenta di creare un	Titolo:	---	---	Viene mostrato un	---	---

	con date non valide (data fine precedent e alla data inizio)	prestito impostando una data di fine precedent e alla data di inizio.	"Prestito DVD" - Data inizio : "2024-07-10" - Data fine: "2024-07-05"			messaggio di errore "Invalid dates" e il prestito non viene creato.		
46	Terminazione di un prestito senza specificare ID	L'utente tenta di terminare un prestito senza fornire un ID valido.	ID Prestito: ""	---	---	Viene mostrato un messaggio di errore "Missing loan ID" e il prestito non viene terminato.	---	---
47	Rinnovo di un prestito senza specificare ID	L'utente tenta di rinnovare un prestito senza fornire un ID valido.	- ID Prestito: "" - Nuova Data Fine: "2024-08-01"	---	---	Viene mostrato un messaggio di errore "Missing loan ID" e il prestito non viene rinnovato.	---	---
48	Recupero dei servizi offerti dal Comune di Trento	L'utente richiede di recuperare tutti i servizi offerti dal Comune di Trento tramite le API.	---	---	---	Viene restituita una lista completa di tutti i servizi disponibili.	---	---
49	Recupero delle notizie con query parameter errati	L'utente richiede di recuperare le notizie utilizzando query	Tipo: "invalidType"	---	---	Viene mostrato un messaggio di errore "Invalid	---	---

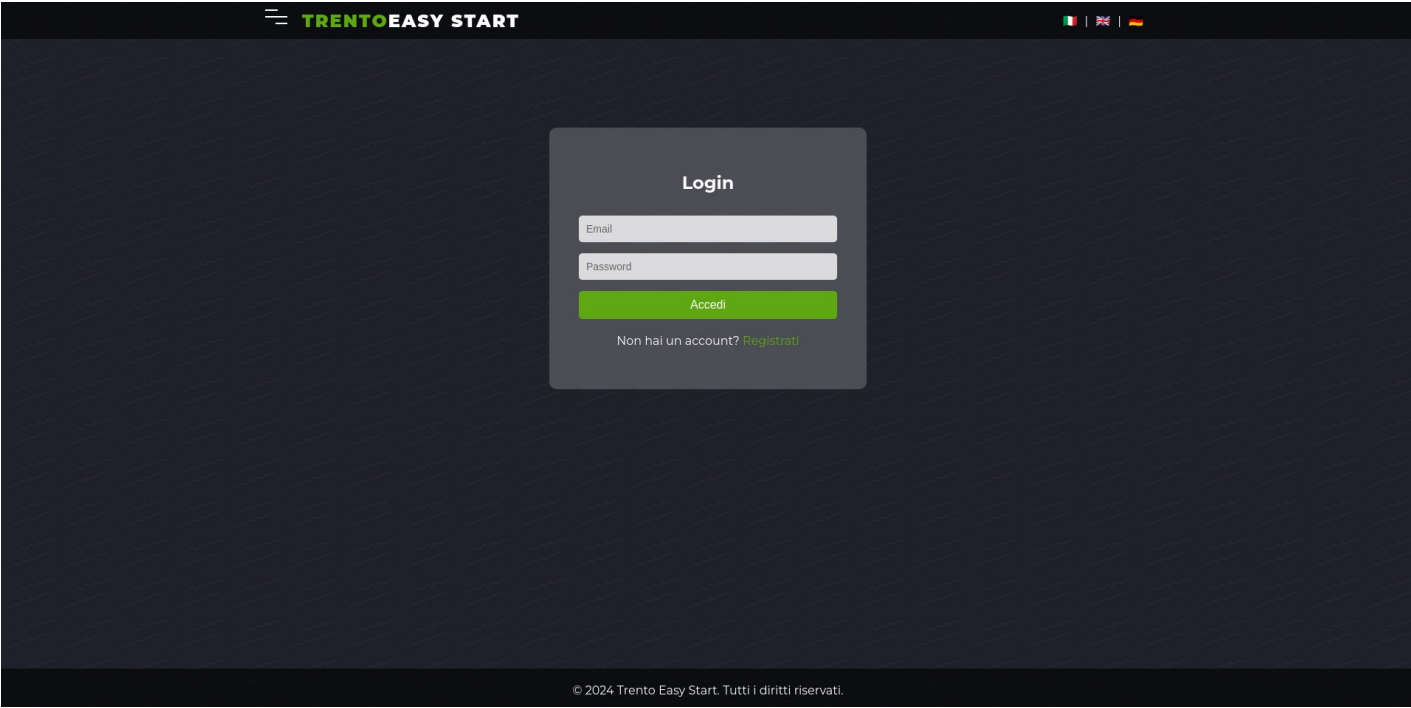
		parameter errati o non supportati .				query paramete rs" e non vengono restituite notizie.		
50	Recupero di un servizio per ID valido	L'utente richiede i dettagli di un servizio specifico utilizzand o un ID valido.	ID Servi zio: "serv ice12 3"	Il servi zio con l'ID speci ficat o esist e.	---	Viene restituito l'oggetto del servizio con tutti i dettagli.	---	---

6. FrontEnd

Il FrontEnd fornisce le funzionalità di visualizzazione, inserimento e gestione dei dati dell'applicazione "Trento Easy Start". L'applicazione è composta da diverse pagine principali:

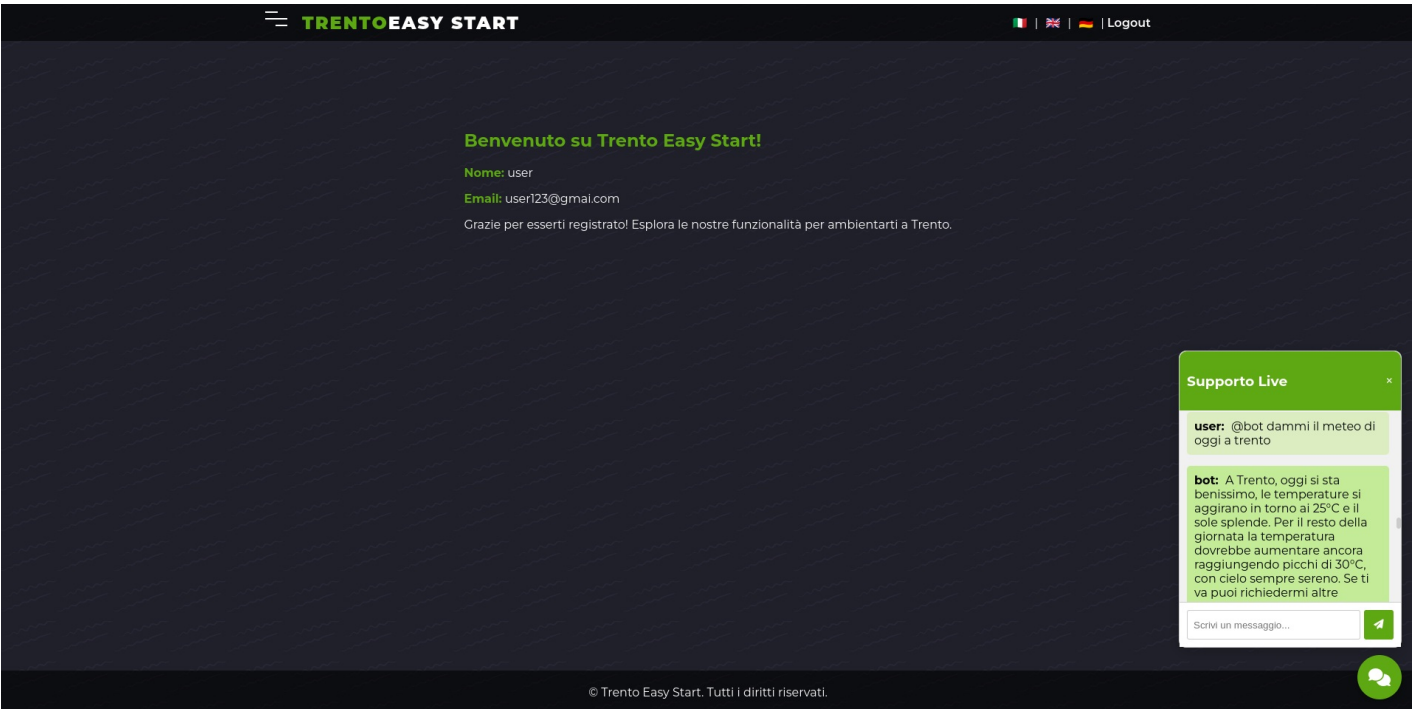
6.1 Login

In Figura 4 è mostrata la pagina di login dell'applicazione. Da qui l'utente può inserire le proprie credenziali per accedere alla piattaforma.



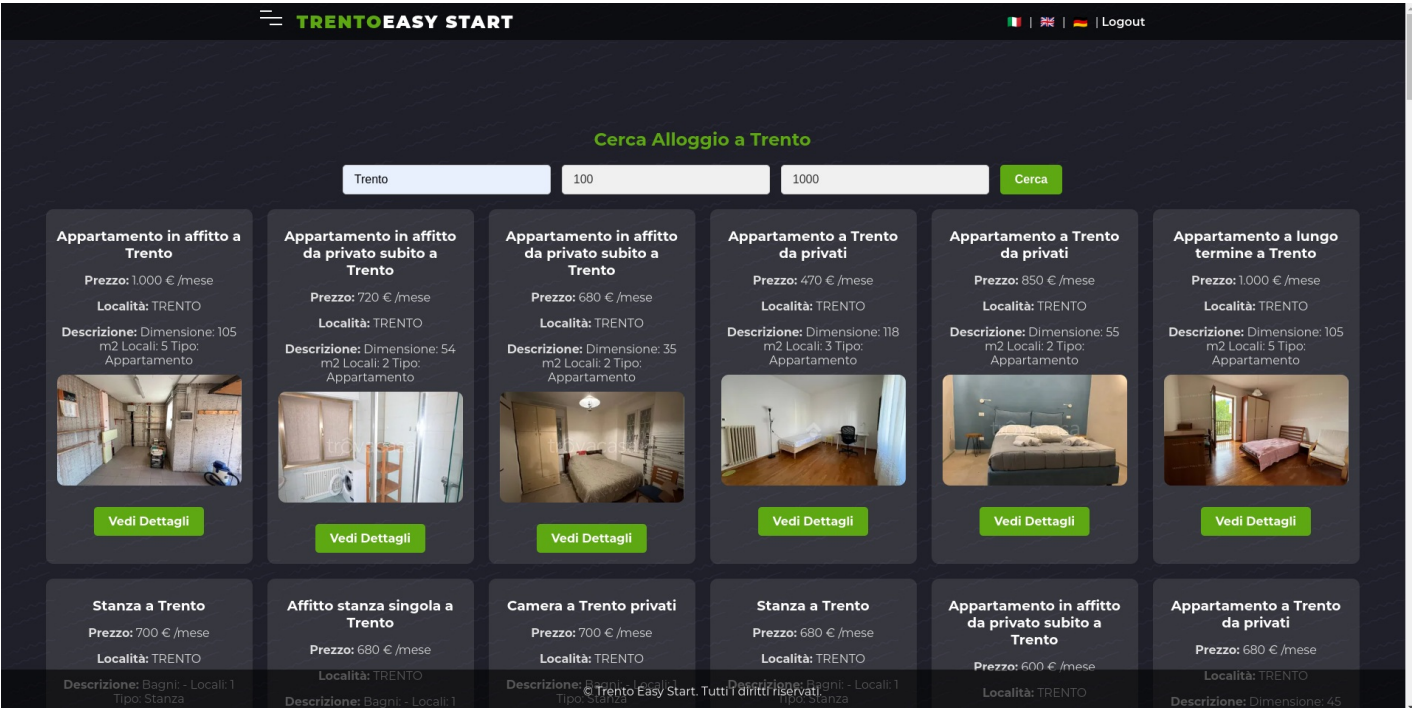
6.2 Main Page

In Figura 5 è mostrata la pagina principale (Main Page) dell'applicazione con dati dell'utente e live chat. Da qui l'utente può navigare verso le varie sezioni disponibili come Cerca Alloggio, Supporto, Servizi, Eventi e Chi Siamo.



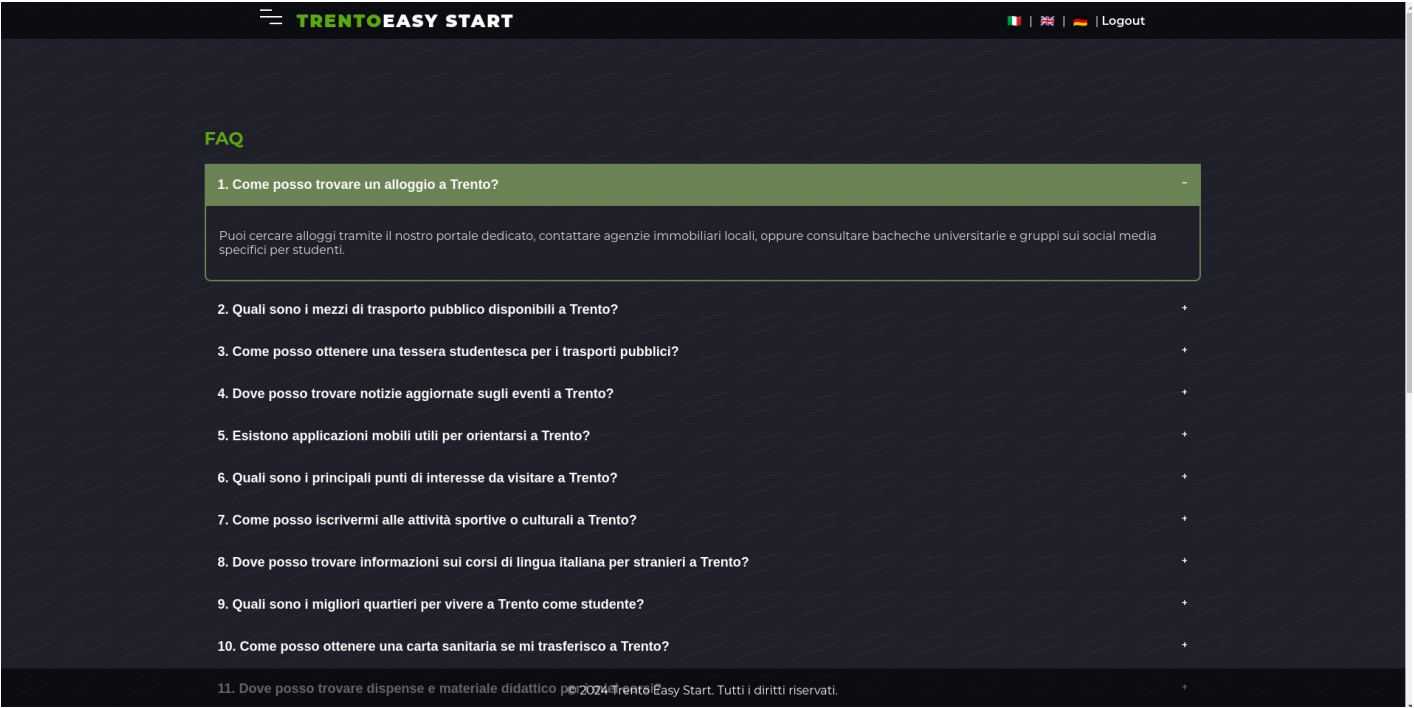
6.3 Cerca Alloggio

In Figura 6 è mostrata la pagina di ricerca degli alloggi. Gli utenti possono filtrare gli alloggi in base a vari criteri come località, tipo e prezzo.



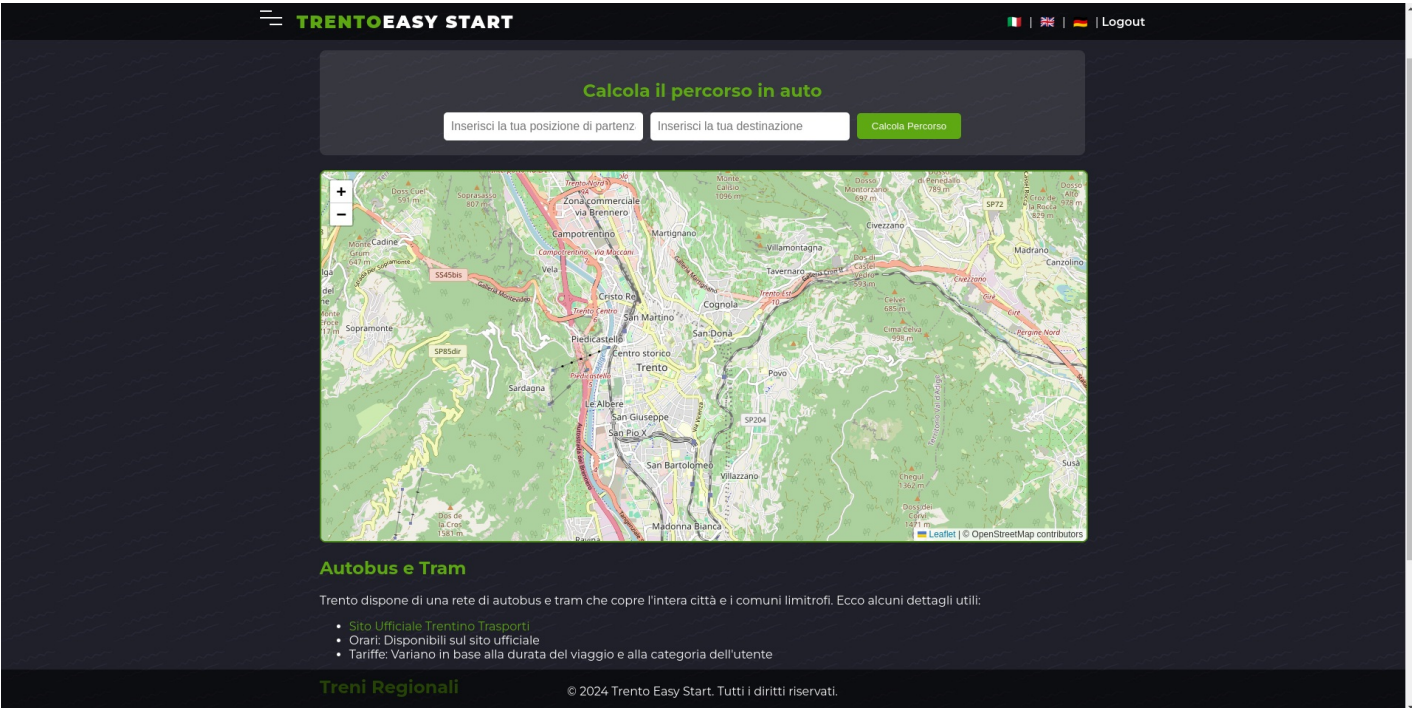
6.4 Supporto

In Figura 7 è mostrata la pagina di supporto. Gli utenti possono inviare richieste di assistenza o consultare le FAQ per risolvere eventuali problemi.



6.5 Servizi

In Figura 8 è mostrata la pagina dei servizi offerti dal Comune di Trento. Gli utenti possono esplorare i diversi servizi disponibili e accedere a ulteriori informazioni.



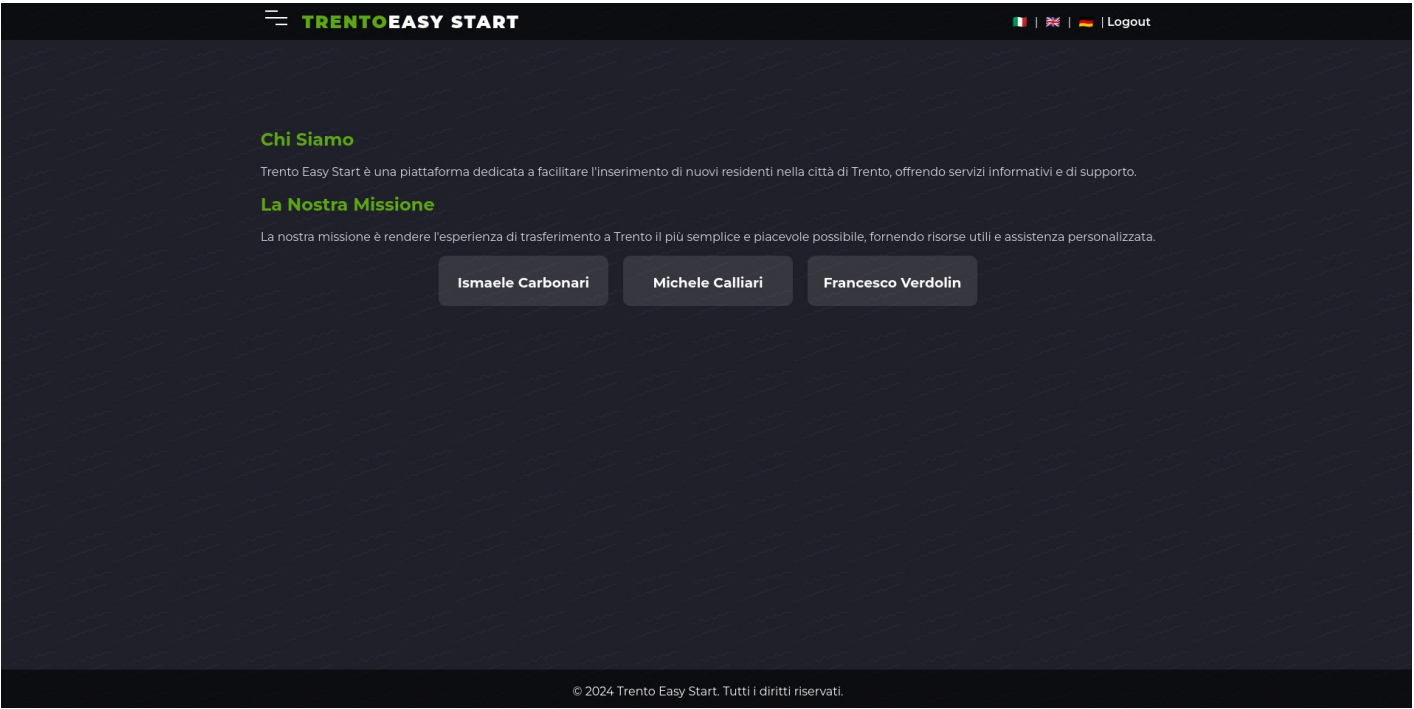
6.6 Eventi

In Figura 9 è mostrata la pagina degli eventi. Gli utenti possono visualizzare gli eventi in programma a Trento, filtrare per categoria e ottenere dettagli su ciascun evento.



6.7 Chi Siamo

In Figura 10 è mostrata la pagina "Chi Siamo". Questa pagina fornisce informazioni sull'organizzazione dietro la piattaforma "Trento Easy Start", i suoi obiettivi e il team di sviluppo.



7. Deployment

Le istanze del backend e del frontend sono ospitate su un Raspberry Pi per garantire disponibilità e scalabilità.

- **Backend e Frontend:** Ospitati su un OnRender e accessibili al link <https://trento-easy-start-1.onrender.com/>.

La configurazione CI/CD basata su GitHub Actions esegue automaticamente il deploy del branch `main` quando vengono superati con successo tutti i test.

Per accedere al deploy dell'applicazione è possibile utilizzare i seguenti account:

- **Utente Studente**

- Username: studente@trentoeasystart.it
- Password: password123

- **Amministratore**

- Username: admin@trentoeasystart.it
- Password: adminpassword

Per problemi con il deploy contattare l'amministratore del sistema all'indirizzo email: admin@trentoeasystart.it.